

# 10. while avec une condition : la boucle qui s'arrête

La condition est posée **AVANT** chaque tour : quand elle devient fausse, on sort.

## 1. Ce qu'il faut comprendre

Un `while` ordinaire a une vraie condition : `while (reste > 0)`. Avant **chaque** tour, la console la calcule. Vraie : elle exécute le bloc, puis revient la poser. Fausse : elle **saute** après l'accolade fermante et continue le programme.

Pour qu'une boucle s'arrête, **quelque chose dans le bloc doit rapprocher la condition du faux**. Ici, c'est `reste--`. Oublie-le, et la boucle tourne pour toujours : l'écran reste bloqué sur 5. C'est la **boucle infinie** involontaire.

Si la condition est fausse dès le départ (essaie `reste = 0`), le bloc ne s'exécute **pas une seule fois** : on passe directement au décollage.

À l'intérieur, une **seconde boucle** fait attendre une demi-seconde : trente tours, un `image()` chacun. Une boucle dans une boucle, c'est permis — on y reviendra.

**À toi** : fais partir le compte à rebours de 9, et accélère-le en attendant 15 images au lieu de 30.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **751 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |       |                                                                       |
|--------------------|-------|-----------------------------------------------------------------------|
| démarrage          | 49 o  | ← la console, à l'allumage — la mise en route de la console           |
| main()             | 79 o  | ← démarrage — ta fonction                                             |
| VBlank             | 8 o   | ← la console, à chaque image — compte les images, 60 fois par seconde |
| AttendreImage      | 28 o  | ← image() — attend l'image suivante                                   |
| AttendreVBlank     | 16 o  | ← démarrage — attend que l'écran soit libre                           |
| AttendreAcces      | 12 o  | ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case       |
| EcrireTexte        | 10 o  | ← texte() — écrit un texte, lettre par lettre                         |
| EcrireNombre       | 33 o  | ← nombre() — écrit un nombre en chiffres                              |
| CopierTuiles       | 76 o  | ← démarrage — copie les lettres et les dessins en mémoire vidéo       |
| EffacerCarte       | 14 o  | ← démarrage — vide les deux cartes de l'écran                         |
| Tuiles             | 400 o | ← CopierTuiles — 25 lettres sur 43                                    |
| "COMPTE A REBOURS" | 16 o  | ← texte() — un texte gravé                                            |
| "DECOLLAGE!"       | 10 o  | ← texte() — un texte gravé                                            |

Lettres gravées : ABCDEGLMOPRSTU0123456789!

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

|          |     |                             |                        |
|----------|-----|-----------------------------|------------------------|
| texte()  | × 2 | 22 o sur place, dans main() | + EcrireTexte (10 o)   |
| nombre() | × 1 | 10 o sur place, dans main() | + EcrireNombre (33 o)  |
| image()  | × 2 | 6 o sur place, dans main()  | + AttendreImage (28 o) |

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un compte à rebours 5, 4, 3, 2, 1, puis DECOLLAGE!  
//  
// Ce qui est nouveau : une boucle while qui S'ARRÊTE.  
// while (condition) ... : répète TANT QUE la condition est vraie.  
// Avant chaque tour, on teste la condition ; dès qu'elle est fausse, on sort.  
  
#include <texte>    // écrit un texte à l'écran  
#include <nombre>  // écrit un nombre en chiffres
```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

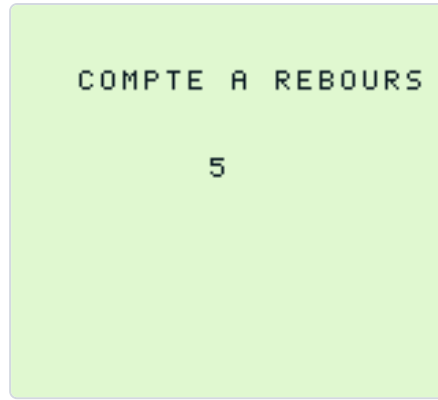
```
int main() {                // Le jeu commence ici.  
    uint8_t reste = 5;      // Le nombre affiché. Il part de 5.  
  
    texte(3, 3, "COMPTE A REBOURS");  
  
    while (reste > 0) {      // Tant que reste est plus grand que 0 :  
        nombre(9, 7, reste, 1); // affiche reste sur 1 chiffre (5, puis 4, ...).  
  
        uint8_t attente = 0; // Un petit compteur pour attendre.  
        while (attente < 30) { // Une boucle DANS la boucle : 30 tours...  
            image();          // ... de une image chacun :  
            attente++;        // 30 images = une demi-seconde.  
        }  
  
        reste--;             // reste perd 1. Sans lui, la boucle ne finit jamais.  
    }                        // Déroulé : 5, 4, 3, 2, 1. Quand reste vaut 0,  
                            // « 0 > 0 » est faux : on sort de la boucle.  
  
    texte(5, 7, "DECOLLAGE!"); // Écrit par-dessus le dernier chiffre.  
  
    while (true) {           // La boucle de fin : on garde l'écran allumé.  
        image();  
    }  
}
```

## 3. Ce qui se passe, étape par étape

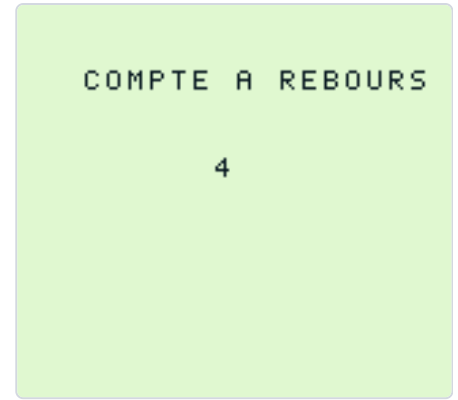
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



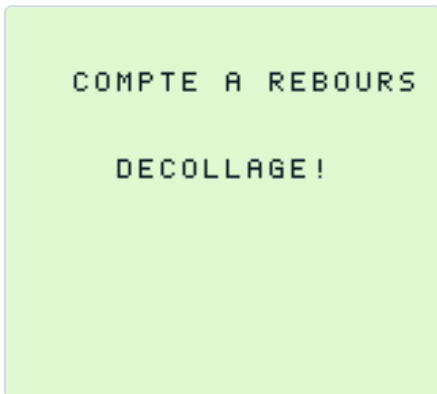
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu'on doit voir

5, 4, 3, 2, 1 toutes les demi-secondes, puis « DECOLLAGE! ».

## 5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on part de 5
- ✓ un peu plus tard, on a décompté (4)
- ✓ la boucle s'est arrêtée, le programme a continué
- ✓ « reste » a bien fini à zéro

← [9. La boucle du jeu : while \(true\).](#)

[sommaire](#)

[11. for : la boucle qui compte](#) →